第6回：意味で探す検索

Embeddingと類似度を理解する

今日のゴールと流れ

文章をベクトルにして、意味の近さで資料を探す仕組みを説明できる。
コサイン類似度を手計算と `for` で求め、しきい値の役割を説明できる。

内容
言い換えに弱いキーワード検索の復習
ベクトル、Embedding、コサイン類似度
完成版 `vector-app/` を動かす・読む
穴埋め：類似度の計算、しきい値
キーワード検索との比較、しきい値の調整、提出

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の進め方

段階	すること
1	`sample-app/vector-app/` の完成版を動かす
2	コードの説明を聞く
3	`session06/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは `sample-app/vector-app/` です。
`vector-app` は `keyword-app` と同じ形で、検索の部分だけをベクトル検索に変えたものです。

言い方が違う質問

質問：「LLMが前の話を忘れるのはなぜ？」
資料：13枚目「LLMは前の会話を覚えていない」

「忘れる」と「覚えていない」は同じ意味ですが、単語としては一致しません。
前回のキーワード検索は同じ単語の数で比べるため、言い換えに弱い面があります。

今回は、Embeddingモデルで文章を数値の列（ベクトル）に変え、向きの近さで比べます。
必ず見つかるわけではありません。実習4でキーワード検索と比較します。

ベクトルとは

数値を並べたものです。説明用の例：

```
a = [1.0, 0.0, 1.0]
b = [0.9, 0.1, 1.0]
c = [0.0, 1.0, 0.0]
```

`a` と `b` は近く、`c` は離れています。
実際のEmbeddingは、多数の次元を持つ長いリストです。
各数値を「価格」「色」のように直接解釈できるとは限りません。

説明用ベクトルの向き

実際のEmbeddingではなく、計算用の2次元の例です。

ベクトル	成分	aとの関係
a	[1, 0]	基準
b	[2, 0]	長さは違うが同じ向き
c	[0, 1]	直交する向き
d	[-1, 0]	反対向き

コサイン類似度では、長さよりも向きの関係を比べます。

生成用と検索用のモデル

用途	モデル	出力
回答を作る	gpt-6-luna	文章など
文章をベクトル化	text-embedding-3-small	数値の列

資料と質問には、同じEmbeddingモデルを使います。
別のモデルが作ったベクトルどうしは、そのまま比較できません。
`main.py` は、DBのベクトルを作ったモデル名を起動時に確かめます。

2つのタイミング

資料の登録時（ init_db.py ）	質問を受けたとき（ main.py ）
全36件の資料をまとめてベクトル化	質問だけをベクトル化
ベクトルとモデル名をDBに保存	DBの各ベクトルと類似度を計算
資料を直すまで使い回す	上位の本文をLLMへ渡す

資料のベクトルを質問のたびに作り直す必要はありません。
サーバーを再起動しても、資料のEmbedding APIは呼びません。

Embedding API

`llm.py` の `embed_texts()` 。文章のリストを、ベクトルのリストにします。

```
def embed_texts(texts, model):
    client = make_client()
    # 複数の文章を1回のAPI呼び出しでまとめてベクトルにする。
    response = client.embeddings.create(model=model, input=texts)
    vectors = []
    for item in response.data:  # 送った文章と同じ順番で返ってくる
        vectors.append(item.embedding)
    return vectors
```

第2回と同じクライアントを使いますが、呼ぶAPIが異なります。
この呼び出しでも外部へ文章を送信し、利用料金が発生します。

コサイン類似度

2つのベクトルの向きの近さを計算します。
内積を、それぞれの長さの積で割ります。

```
内積 = a[0]×b[0] + a[1]×b[1] + …
長さ = 各成分の2乗の合計の平方根
類似度 = 内積 ÷ (aの長さ × bの長さ)
```

同じ向きは1、直交は0、反対向きは-1です。
ゼロベクトルは向きが無いので、教材では0として扱います。

同じ向きの計算例

a = [1, 0]、b = [2, 0] の場合：

計算	結果
内積	$1 \times 2 + 0 \times 0 = 2$
aの長さ	$\sqrt{(1^2 + 0^2)} = 1$
bの長さ	$\sqrt{(2^2 + 0^2)} = 2$
コサイン類似度	$2 \div (1 \times 2) = 1$

長さが2倍でも、同じ向きなら類似度は1です。

向きが異なる計算例

a = [1, 0] を基準にします。

比較相手	内積	長さの積	類似度
[0, 1]	0	1	0
[-1, 0]	-1	1	-1
[0, 0]	0	0	教材では0

ゼロベクトルは長さが0なので、そのままでは割り算できません。
`cosine_similarity()` は、どちらかの長さが0なら `0.0` を返します。

forで内積を計算

`search.py` の `cosine_similarity()` の中心部分です。

```
    dot = 0.0     # 内積: 同じ位置の数どうしを掛けて、全部足したもの
    a_length = 0.0  # a の長さの2乗
    b_length = 0.0  # b の長さの2乗
    for i in range(len(a)):
        dot += a[i] * b[i]
        a_length += a[i] * a[i]
        b_length += b[i] * b[i]
    if a_length == 0 or b_length == 0:
        return 0.0  # 長さ0のベクトルは向きが無いので、0とする
    return dot / (math.sqrt(a_length) * math.sqrt(b_length))
```

`i` はリストの位置です。同じ位置の成分どうしを掛けます。

教材では数値計算ライブラリを使わず、処理を追えるようにしています。

ループの途中の値

a = [1, 2]、b = [2, 1] で計算します。

段階	dot	a_length	b_length
初期値	0	0	0
i = 0 の後	2	1	4
i = 1 の後	4	5	5

類似度は $4 \div (\sqrt{5} \times \sqrt{5}) = \mathbf{0.8}$ です。

`a_length` と `b_length` は、ループの中では長さの2乗です。

`math.sqrt()` で平方根にするのは、割り算の直前です。

実習1：完成版を動かす

```
source .venv/bin/activate
cd sample-app/vector-app
python init_db.py
python main.py
```

`init_db.py` は36件の資料を1回のAPI呼び出しでまとめてベクトルにします。
APIキーが必要で、利用料金が発生します。実行前に次の文が表示されます。
「36個の資料をEmbedding APIへ送り、ベクトルにします。利用料金が発生します。」

「ポート」タブから8002番を開き、「学習と推論の違いは？」と質問します。
渡された資料ごとに、類似度が小数第3位まで表示されます。

今回のファイル

ファイル	役割
`init_db.py`	スライドを資料に分け、ベクトルにして `documents.db` に保存
`main.py`	起動時にDBを読む。`ask()` が質問を受けて3段階で処理
`llm.py`	`embed_texts()` と `ask_with_documents()`
`search.py`	`cosine_similarity()` と `search_documents()`
`static/`	画面。`keyword-app` とほぼ同じ

`keyword-app` と違うのは、単語の代わりにベクトルを保存し、一致数の代わりに類似度で並べる点です。

読む順番は `init_db.py` の `add_vectors()`、`main.py`、`search.py` です。

init_db.py：資料をベクトルにする

```
def add_vectors(documents, embedding_model):
    """全部の資料をEmbedding APIでベクトルにし、各資料の 'vector' に入れる。"""
    texts = []
    for document in documents:
        texts.append(document['title'] + '\n' + document['heading'] + '\n' + document['body'])
    # 1回のAPI呼び出しで、全部の資料をまとめてベクトルにする。
    vectors = embed_texts(texts, embedding_model)
    for i in range(len(documents)):
        documents[i]['vector'] = vectors[i]  # 資料と同じ順番でベクトルが返ってくる
```

題名・見出し・本文をつないだ文章を、資料1件につき1つ作ります。
36件の文章をリストにして、`embed_texts()` に1回で渡します。

DBに保存するもの

`init_db()` は `documents` テーブルを作り直し、1行に1資料を保存します。

列	保存する理由
`body`（と題名・見出し）	LLMへ渡すのは元の文章
`vector`	次の質問で再利用する。`json.dumps()` でJSONの文字列にする
`embedding_model`	質問と同じモデルか確かめる
`source` ・ `slide`	原本の何枚目かを確かめる

SQLiteにリストの型は無いので、ベクトルは文字列にして保存します。
ベクトルだけをLLMへ渡しても、回答は作れません。本文が必要です。

main.py：起動時にDBを読む

```
def load_documents(database, embedding_model):
    if not database.exists():
        raise SystemExit(f'{database.name} がありません。先に python init_db.py を実行してください。')
    ...
    for row in rows:
        # 別のモデルで作ったベクトルとは比べられないので、作り直してもらう。
        if row[7] != embedding_model:
            raise SystemExit(f'DBのベクトルは {row[7]} で作られています。'
                             f'python init_db.py を実行して、{embedding_model} で作り直してください。')
```

DBの `embedding_model` が `EMBEDDING_MODEL` と違えば、起動時に止まります。
`vector` の列は `json.loads(row[6])` で、JSONの文字列から数値のリストに戻します。

main.py：ask() の3段階

```
# 1. 質問をベクトルにする。資料と同じモデルを使う。
question_vector = embed_texts([question], EMBEDDING_MODEL)[0]

# 2. 質問のベクトルに近い資料を探す。
found = search_documents(question_vector, documents, TOP_K, MIN_SIMILARITY)

# 3. 見つかった資料を渡して、LLMに回答させる。
if not found:
    # 資料が無いときは、回答用のAPIを呼ばずに知らせる。
```

定数	値	意味
TOP_K	3	LLMに渡す資料の最大件数
MIN_SIMILARITY	0.3	これより類似度が小さい資料は渡さない

質問1つにつき、Embedding APIを1回と、見つかれば回答のAPIを1回呼びます。

search.py：近い資料を選ぶ

```
def search_documents(question_vector, documents, top_k, min_similarity):
    results = []
    for document in documents:
        score = cosine_similarity(question_vector, document['vector'])
        if score >= min_similarity:
            results.append({
                ...
                'score': score,
            })
    # 類似度の大きい順に並べる。
    results.sort(key=get_score, reverse=True)
    return results[:top_k]
```

全資料との類似度を計算し、しきい値以上だけ残し、大きい順に3件までです。
ベクトルそのものは返しません。画面に出すのは `score` です。

スコアの読み方

画面の「類似度」は `score` を小数第3位まで表示したものです。

- 1に近いほど、質問と資料の向きが近い
- 類似度は、回答が正しい確率ではない
- キーワード検索の点数（一致した単語の数）と同じ尺度ではない
- 高い類似度でも、質問の答えが本文に無い場合がある

`MIN_SIMILARITY` の0.3は目安です。画面に出る類似度を見て調整します。
回答に付いた資料IDと本文を見比べて、根拠を確かめます。

実習2：類似度の計算を埋める

`session06/exercise/search.py` の `cosine_similarity()` を完成させます。
先に、埋める前の動きをターミナルで確かめます。APIキーは不要です。

```
cd session06/exercise
python
```

```
from search import cosine_similarity
cosine_similarity([1, 0], [1, 0])  # 埋める前は 0.0
```

`a_length` が0のままなので、同じ向きでも `0.0` が返ります。
`exit()` で抜け、`TODO` を探して `for` の4行のコメントを外します。
もう一度 `python` で同じ計算をすると `1.0` になります。

計算結果の確認

埋めた後に、次の値も確かめます。

入力	期待する結果
[1, 0] と [2, 0]	1
[1, 0] と [0, 1]	0
[1, 2] と [2, 1]	約0.8
[0, 0] と [1, 0]	教材の取り決めで0

小数の計算では、表示の末尾に丸めの差が出ることがあります。
手計算の表（同じ向き・向きが異なる・ループの途中）と一致することを確認します。

実習3：しきい値を埋める

`search_documents()` の `TODO` を探します。仮の行は `if score > 0:` です。
先に、完成版のサーバーを止めてから、仮の行のまま動かします。

```
cd session06/exercise
python init_db.py  # exercise 側にも documents.db を作る（料金が発生）
python main.py
```

8002番で「校庭の桜はいつ咲く？」と質問します。資料に無い話題です。
しきい値が効かないので、類似度が低くても上位3件が渡され、回答が作られます。
画面の類似度が0.3未満の資料が混ざっていることを確かめます。

Ctrl+Cで止め、`# if score >= min_similarity:` のコメントを外し、仮の `if` の行を消します。
`python main.py` で再起動し、同じ質問で類似度0.3未満の資料が渡されないことを確かめます。
0件なら「関係する資料が見つかりませんでした。」と表示され、回答のAPIは呼ばれません。

実習4：キーワード検索と比較

ターミナルをもう1つ開き、キーを設定して `keyword-app` （8001番）も動かします。

```
export OPENAI_API_KEY=sk-********
source .venv/bin/activate
cd sample-app/keyword-app
python init_db.py  # documents.db が無いときだけ。APIキーは不要
python main.py
```

8001番と8002番の両方で、同じ3つの質問をします。

1. 「学習と推論の違いは？」
2. 「LLMが前の話を忘れるのはなぜ？」
3. 「来年度の試験日は何月何日？」

比較の記録

質問ごとに、選ばれた資料と数値を記録します。

質問	キーワード検索（資料・点数）	ベクトル検索（資料・類似度）
学習と推論の違いは？		
LLMが前の話を忘れるのはなぜ？		
来年度の試験日は何月何日？		

- 13枚目「LLMは前の会話を覚えていない」が、どちらで選ばれたか
- 「来年度の試験日は何月何日？」は第1回の32枚目に質問文があるので、どちらでも32枚目が見つかる。回答が試験日を断定しないか
- 点数と類似度は尺度が違うので、数値の大小で優劣を決めない

実習5：しきい値を変えてみる

`session06/exercise/main.py` の `MIN_SIMILARITY` を変え、サーバーを再起動します。

値	予想
0.5	渡る資料が減る。0件になる質問が増える
0.3	元の値
0.2	関係の薄い資料まで渡ることがある

同じ3つの質問で、渡る件数と類似度を見比べます。
ベクトルは変わらないので、`init_db.py` のやり直しは不要です。
試した後は `0.3` に戻します。

init_db.py をやり直すとき

変えたもの	init_db.py	理由
data/ の資料	必要	資料の本文とベクトルを作り直す
EMBEDDING_MODEL	必要	別のモデルのベクトルとは比べられない。main.py が止まる
MIN_SIMILARITY ・ TOP_K	不要	検索のときに使う値。再起動だけでよい
search.py の穴埋め	不要	計算の仕方だけが変わる

init_db.py は実行するたびに documents テーブルを作り直し、Embedding APIを呼びます。
documents.db はGitには入れません。

費用と規模

場面	呼ぶAPI
`init_db.py` の実行	Embedding API 1回（36件の資料をまとめて）
質問1回	Embedding API 1回（質問）＋ 回答のAPI 1回
資料が0件の質問	Embedding API 1回のみ

この教材は36件の資料を、Pythonの `for` で全件比較します。
資料が数万件になると、全件比較では時間がかかります。
大規模な検索の基盤は発展課題とし、まずは検索精度と資料の整え方を学びます。

失敗した場所の切り分け

観察した状態	まず確認すること
「documents.db がありません」	そのフォルダで `python init_db.py` を実行したか
「DBのベクトルは … で作られています」	`EMBEDDING_MODEL` を変えた。`init_db.py` をやり直す
「環境変数 OPENAI_API_KEY を設定して…」	同じターミナルで `export` したか。`init_db.py` でも必要
どの質問でも「関係する資料が見つかりません」	実習2の `for` を外したか（類似度が常に0）
関係の薄い資料が3件渡される	実習3の仮の `if` を消したか
8002番で起動できない	完成版のサーバーを止めたか

相談時は、操作手順とエラーの種類を伝えます。キーは共有しません。

確認問題

1. 資料と質問に異なるEmbeddingモデルを使ってよい？
2. 類似度0.8は正答率80%？
3. サーバーを再起動したとき、資料のEmbedding APIは呼ばれる？
4. `MIN_SIMILARITY` を変えたとき、`init_db.py` のやり直しは必要？

提出物と次回

`session06/exercise/` をコミットし、プッシュします。 `documents.db` は入れません。

```
cd ../..  # リポジトリのルートへ戻る
git add session06/exercise
git commit -m "第6回: ベクトル検索"
git push
```

GitHubで `session06/exercise` を開き、そのURLをフォームから提出します。
サーバーを止め、両方のターミナルで `unset OPENAI_API_KEY` を実行します。

次回は、文章の検索ではなく、DBの値を関数で取得します。